DELIVERABLE 2 REPORT — ADC SUBTEAM 46

Authors: Leo Jiang and Caleb Vaillancourt

Contents

1. Introduction and System Context 3
2. Precise Specification 4
2.1 System Boundary and Four-Variable Model 4
2.2 NAT and REQ Relationships 5
2.3 Precise Functional Specifications 6
2.4 Preconditions, Postconditions, and Invariants 13
2.5 Coverage of D1 Requirements 16
3. Modular Design and Interfaces 17
3.1 Hardware-Hiding Modules 17
3.1.1 Cabinet Motor and Sensor Driver 17
3.1.2 Serial Driver 18
3.2 Behaviour-Hiding Modules 19
3.2.1 Cabinet Controller 19
3.2.2 Authenticator 22
3.2.3 Inventory Tracker 24
3.2.4 External Communication Driver 25
3.3 Common Services 27
3.3.1 Event Logger 27
3.4 External Interface to the Monitoring System 28
Provides 28
Requires 30
Protocol Secrecy 30
Degraded behaviour 31
4. Implementation Notes 32
Architecture diagram to code 32
State machine 34
Activity flow 36
Use cases to code 37
5. Traceability Table 37
5.1 Requirement verification and implementation status 42
5.2 Automated 1,000-case inventory-accuracy test (VT-INV-01) 43
5.3 Remaining quantitative targets 44

6. Assumptions and Decisions 44
Citations.....47

1. Introduction and System Context

The goal of this project is to develop the software for an Automated Dispensing Cabinet (ADC) and its associated medication monitoring system for use in a hospital environment. This report focuses on the design and implementation of the ADC firmware component, which coordinates the cabinet's internal software modules, including authentication, inventory management, event logging, and external communication

Building upon the requirements established in Deliverable 1, this report presents a more detailed specification of the system's expected behaviour and documents the modular architecture used to implement it. Particular attention is given to defining module responsibilities, establishing clear interfaces between the components, and implementing information hiding to improve maintainability and make future modifications easier to implement.

Since D1, the ADC design has been refined to match the seven-state controller, the Monitoring System-initiated serial protocol, bounded inventory storage, and explicit failure handling. The table identifies changes by original requirement ID; the full D1-to-precise-specification mapping remains in Section 2.5.

Table 1.1: Summary of requirements revised or added since D1

D1 ID	D1 wording (short)	Change in D2	Reason
FR-INV-02	Cabinet notifies the monitoring system when restocking is needed	Monitoring System requests the inventory and decides; the ADC only answers (PS-INV-04)	Monitoring System is master, ADC is slave; one place decides restocking
FR-EXT-01	Cabinet sends dispensing, inventory and fault information	Monitoring System requests this information; the ADC never sends unasked (PS-EXT-01)	Same master and slave decision
FR-FAU-01, NFR-SEC-02	Notify IT after 3 failed attempts; no access	The lockout applies to the user, for 60 s; the count is readable by the Monitoring System (PS-AUT-02)	One user's mistakes must not lock out the whole ADC for every nurse
New FR-INV-03	None	Max 100 units per medicine and 300 units per cabinet (PS-INV-02, PS-INV-03)	Physical capacity of a cabinet
FR-OPE-01	Four modes	Seven states, listed in Table 2.3	Matches the controller; Maintenance, Fault and Restocking are added
FR-AUT-01	Staff authentication	Three user levels, and an allowed-medicine list per nurse (PS-AUT-03)	Controlled access and patient safety

D1 ID	D1 wording (short)	Change in D2	Reason
FR-EVE-01	Events displayed on the UI	Events go to a log file, available to the Monitoring System, not to nurses (PS-EVE-01)	Patient privacy; see Assumptions and Decisions
FR-DIS-01	Open the drawer	The term "drawer" becomes "cabinet"; the most recent inventory delta is rolled back if opening fails (PS-DIS-01)	Matches the code; the failed opens must not be recorded as successful transactions

The changes are reflected in the requirements (Section 2), module contracts (Section 3), code mappings (Section 4), and traceability/verification evidence (Section 5). Planned functionality is identified separately from behaviour described as implemented.

2. Precise Specification

2.1 System Boundary and Four-Variable Model

The ADC firmware operates between the physical cabinet hardware, the user interface, and the external hospital monitoring system. It receives information from users, sensors, and the monitoring system, processes this information through its internal software modules, and produces outputs that control cabinet operations or communicate system information externally.

To establish a precise specification of the firmware, the system is described using the four-variable model. This model identifies the monitored variables, controlled variables, software inputs, and software outputs that define the interaction between the ADC firmware and its environment.

Variable Category	Description	ADC Examples
Monitored Variables (M)	Conditions in the environment that the firmware needs to observe.	Observed physical drawer position, sensor-indicated jam/fault, and physical link availability. Physical medication count is not sensed by this prototype.
Controlled Variables (C)	Conditions in the environment that the firmware is intended to influence.	Physical drawer position/access state and locking/closing motion influenced by motor commands; software inventory records are internal state, not a physically measured C variable.
Inputs (I)	Information received by the firmware from external sources.	Position/fault sensor readings, trusted card-reader identity assertion, typed login/dispense/restock requests, and Monitoring System messages.
Outputs (O)	Information or commands sent from the firmware to external components.	Open/close motor commands, UI result/status messages, and serial responses. Inventory/log file writes are internal effects, not physical outputs.

The boundary distinguishes physical facts from their representation. A drawer being physically open is a monitored environmental fact (M), its position sensor reading is a software input (I), a

close motor command is an output (O), and the resulting drawer position is a controlled environmental condition (C). The prototype does not measure the physical count of medication; inventory quantities are an internal ledger updated from accepted transactions. Its accuracy claim therefore concerns ledger consistency, not a verified physical medication count.

Table 2.1a instantiates the four-variable model. NAT relates environmental conditions to sensor inputs and actuator outputs; REQ relates monitored circumstances to required physical outcomes. The software specification (SOF) maps received inputs to outputs so that, subject to NAT, REQ is satisfied.

M: monitored/environment	**I: software input**	**O: software output**	**C: controlled/environment**
Drawer is physically OPEN/CLOSED or jammed	Sensor returns open/closed/fault (NAT-2, NAT-3)	open_cabinet(c), close_cabinet(c)	Drawer moves to requested position or remains faulted (REQ-1, REQ-4)
Authorized nurse requests access	Verified card identity or credential submission (NAT-4, NAT-7)	Access result and permitted motor command	Only authorized drawer access (REQ-1, REQ-7)
Communication line connected/disconnected	Serial link status and received request (NAT-6)	Exactly one response when exchange completes	Monitoring System observes the ADC state after polling (REQ-6)

2.2 NAT and REQ Relationships

Natural relations (assumptions about the environment)

ID	Natural relation	Used by
NAT-1	A cabinet is in one physical state at a time (OPEN or CLOSED). A closed cabinet cannot be opened by hand.	PS-AUT-01, PS-DIS-01
NAT-2	After a motor command, a cabinet reaches its new position within T_motor = 3 s (hardware limit, to be confirmed with the real motor). The sensor shows the new position only once the movement has finished.	PS-DIS-01, PS-FAU-02
NAT-3	A cabinet mechanism can jam. The sensor then reports "fault" or a position that differs from the commanded one.	PS-FAU-02
NAT-4	The physical card reader completes card decryption, cryptographic verification and authenticity checks before it sends a trusted verified username to the ADC. An unverified username must not enter through the card-login path. The username may still be absent from the local user database.	PS-AUT-01, PS-AUT-02
NAT-5	Medicine can only be taken from or put into a cabinet that is OPEN.	PS-DIS-01, PS-INV-05
NAT-6	The serial link can drop at any time. A message sent while it is down is lost.	PS-EXT-02

ID	Natural relation	Used by
NAT-7	The card-reader interface is trusted only at the hardware boundary; keyboard, GUI and arbitrary serial messages are not equivalent to a verified card event.	PS-AUT-01, PS-AUT-02
NAT-8	No physical medication-count sensor exists in the current prototype. Quantities are derived from persisted, authorized dispense/restock transactions.	PS-INV-04, PS-INV-06

Required relations (what the ADC must do)

ID	Required relation (monitored to controlled)	D1 IDs
REQ-1	A cabinet is opened only for a logged-in Nurse, in Dispensing or Restocking state, for a medicine on that nurse's allowed list.	FR-AUT-01, NFR-SEC-01
REQ-2	Under normal motor and sensor operation (NAT-2), a valid confirmed dispense opens the requested drawer within 5 s; on a failed open, return ERROR and restore the most recent inventory change without reporting SUCCESS.	FR-DIS-01, NFR-PER-01
REQ-3	After 3 consecutive failed password attempts by one user, that user gets no session for 60 s.	FR-FAU-01, NFR-SEC-02
REQ-4	At the first monitor check at or after 180 s of continuous OPEN (no later than 181 s given 1 s sampling), command CLOSE and enter Fault; a failed close keeps the system in Fault and is logged.	FR-FAU-02
REQ-5	A cabinet that reports "fault" receives no dispense or restock requests, and the Monitoring System can read that fault.	FR-FAU-03
REQ-6	While the link is up, a request for the inventory is answered with the current quantity of every medicine in every cabinet.	FR-INV-02, FR-EXT-01
REQ-7	In Fault, Locked and EmergencyShutdown state, no dispense or restock request is accepted.	FR-OPE-01
REQ-8	Logout shall not clear Locked, Fault or EmergencyShutdown. A transition from Fault/EmergencyShutdown to Standby requires an authorized recovery request after the unsafe condition has cleared.	FR-OPE-01, NFR-SEC-01
REQ-9	If a motor open command fails after an inventory update, the most recent transaction delta is rolled back before ERROR; no unrelated committed inventory changes are undone.	FR-DIS-01, FR-INV-01, NFR-REL-01

2.3 Precise Functional Specifications

The original functional and non-functional requirements from Deliverable 1 have been refined into more specific software specifications. These specifications define the expected behaviour of the ADC firmware and provide a basis for implementation and testing.

ID	D1 source	Specification
PS-AUT-01	FR-AUT-01, NFR-SEC-01 US-SEC-01	The ADC shall accept a username/password login or a verified card-reader username. The card path may be invoked only by the trusted physical reader after it has decrypted and authenticated the presented card (NAT-4, NAT-7); an arbitrary typed username is not evidence of a verified card. Unknown usernames return NOT_FOUND and wrong passwords PASSWORD_INCORRECT. Only one user may be logged in; a second attempt returns USER_ALREADY_LOGGED_IN without increasing failed attempts. Valid credentials for a role not permitted in the current state (Table 2.2) return NOT_ALLOWED_IN_STATE without creating a session. No cabinet may open, and no inventory may be shown, before a valid session exists.
PS-AUT-02	FR-FAU-01, NFR-SEC-02 US-SEC-02	The ADC shall count consecutive failed password attempts per username and persist the count and time. Success resets the count. After 3 consecutive incorrect passwords, subsequent password logins return LOCKED_OUT until 60 s after the third failure; attempts during the lockout do not extend it. Afterwards, a fresh set of 3 attempts is permitted. A card event already decrypted and verified by the trusted physical reader is outside this password lockout and resets the count; this exemption must not apply to untrusted username-only input. Each failure and lockout is logged, and the Monitoring System may read the count and timestamp via PS-EXT-01.
PS-AUT-03	New (user levels) US-SEC-01, US-FAU-02, US-EVE-02	Each user shall have one level and a list of allowed medicines. Table 2.2 gives what each level may do. A Nurse may only dispense or restock medicines on their own list. Admin and Maintenance shall never open a cabinet to handle medicine. Users shall be added, edited and deleted through the controller or by the Monitoring System. A user who is deleted while logged in shall be logged out.

ID	D1 source	Specification
PS-DIS-01	FR-DIS-01, NFR-PER-01 US-DIS-01, US-PER-01	A dispense(c,m,q) is valid only for an authenticated Nurse in Dispensing; c is an integer 1..N (decimal digit strings accepted); m is a nonempty, case-insensitive name on that Nurse's allowed list and held in c; c is not faulted; q is an integer with 1 <= q <= stock(c,m). The controller records the pending inventory decrement for this request, applies exactly that decrement, then commands drawer c to open. If the motor reports successful opening within 5 s, the request returns SUCCESS and is logged. If opening fails, the controller rolls back only this request's most recent inventory delta (-q), restores the prior quantity, logs the failed open/rollback, and returns ERROR; unrelated committed changes remain intact. If rollback itself fails, the discrepancy is logged at ERROR, the cabinet is treated as faulted, and SUCCESS is prohibited. Invalid requests leave all quantities unchanged and return NOT_LOGGED_IN, ACCESS_DENIED, NOT_ALLOWED_IN_STATE, INVALID_VALUE, NOT_FOUND, or REJECTED as appropriate. No partial dispense is permitted. The drawer remains open until closed or the user logs out, subject to PS-FAU-01.
PS-INV-01	FR-INV-01 US-INV-01, US-EVE-01	The inventory shall hold one record for each (cabinet, medicine) pair: cabinet (integer 1..N), medicine name (stored in lower case) and quantity (integer ≥ 0). A logged-in user of any level shall be able to read the inventory, grouped by medicine with the total and the quantity in each cabinet; medicines with quantity 0 shall be left out of that view.
PS-INV-02	New FR-INV-03	The quantity of one medicine in one cabinet shall never exceed 100 units.
PS-INV-03	New FR-INV-03	The total quantity of all medicines in one cabinet shall never exceed 300 units.
PS-INV-04	FR-INV-02, NFR-REL-01 US-EXT-01	The ADC shall not decide when restocking is needed. The Monitoring System shall poll stored inventory (optionally filtered by cabinet/medicine) and evaluate its own low-stock threshold. Responses contain all matching stored records. Each inventory change is serialized under a lock and persisted by writing a temporary file then atomically replacing the stored file, so readers cannot see partially written records. Because the prototype has no physical count sensor (NAT-8), NFR-REL-01 accuracy is measured against a reference transaction ledger by automated test VT-INV-01: at least 990 of 1,000 independently checked transaction outcomes must match, and no safety-bound or rollback invariant may fail.

ID	D1 source	Specification
PS-INV-05	FR-INV-01 US-USE-03	A restock(c,m,q) obeys the PS-DIS-01 authentication, allowed-medicine and drawer-fault guards, but requires Restocking and a positive integer q with stock(c,m)+q <= 100 and total(c)+q <= 300. Requests exceeding either limit are rejected with no change or partial restock. On acceptance the controller persists exactly the pending +q transaction delta, then commands c to open. If opening fails, the most recent +q delta for this request is rolled back, restoring the prior quantity while leaving other transactions untouched; the failure and rollback are logged and ERROR is returned. If the rollback cannot be persisted, mark a fault and never report SUCCESS.
PS-INV-06	NFR-REL-01 (99% accuracy)	Software inventory-record accuracy shall be verified by automated test VT-INV-01 with 1,000 deterministic randomized transactions (valid dispense, restock, rejected requests, open failures and restart/readback). Each case compares stored per-(cabinet,medicine) quantities with an independently maintained expected ledger. Accuracy = matching cases / 1,000; the D1 99% criterion requires >= 990 matches. All failed-open rollback checks, nonnegative stock and capacity invariants must pass in every case. The metric is ledger accuracy; physical stock counting is not claimed.
PS-EVE-01	FR-EVE-01, NFR-TRA-01 US-EVE-01, US-EVE-02	The ADC shall append one line to the event log for each of: login success, login failure, lockout, logout, state change (with the user or "controller" that caused it), dispense, restock, cabinet open and close, rejected request (with the reason), fault, and communication start, stop and each message sent. Each line shall hold the date and time (to the millisecond), the source, a severity (DEBUG, INFO, WARNING or ERROR) and a message. The message of a dispense or restock line shall name the user, medicine, quantity and cabinet. The log shall be kept in a file that survives a restart. The log shall not be readable by a Nurse through the ADC; it is available to the Monitoring System (PS-EXT-01).

ID	D1 source	Specification
PS-EXT-01	FR-EXT-01, FR-INV-02 US-EXT-01, US-FAU-01	The ADC shall be the slave on the serial link (115200 baud, 8 data bits, no parity, 1 stop bit, one message per line). The Monitoring System shall send a JSON object with device_id, type ("get" or "set"), target and optional params. For every request the ADC shall send exactly one response with device_id, type "response", target, status, status_name, and optionally field and data. A line that is not a JSON object, a request with a missing field, or a type other than get or set shall return INVALID_REQUEST. A device_id that is not this ADC's shall return WRONG_DEVICE ("*" is accepted only for get controller_state). An unknown target shall return UNKNOWN_TARGET. The targets are: auth_details (get, set add/edit/delete), inventory (get), controller_state (get: state, cabinet states, open time), events (get). For a complete request received while the link is Ready and with at most one outstanding request, the response shall be sent within 1 s of receipt; if the link fails during the exchange, the Monitoring System treats the request as timed out. The ADC shall never send a message unasked: lockouts and faults are found by polling controller_state and auth_details.
PS-EXT-02	FR-EXT-01 US-FAU-01	The ADC provides to the Monitoring System: the inventory, the state, the cabinet states, the user list (without passwords) and the event log. It requires from the Monitoring System: requests in the format of PS-EXT-01, and a repeat of any request that gets no response. When the link is down the ADC shall (a) keep running locally: login, dispensing, restocking, fault monitoring and logging do not depend on the link; (b) not process requests until the link is back. Messages received are held in a queue of 100; when it is full a new message shall be dropped, a warning logged and no response sent. The ADC shall never answer SUCCESS to a request it has not carried out.
PS-FAU-01	FR-FAU-02 US-SEC-01	The ADC shall sample every drawer at least once per second. On the first observation of 60 s continuously OPEN, log exactly one WARNING. On the first observation at or after 180 s (within 1 s of that threshold), log exactly one ERROR, issue a close_cabinet command, and enter Fault even if the close command fails. Log the close result and keep the drawer faulted when physical closure cannot be confirmed. A later closed-to-open transition resets the open timer. The Monitoring System reads the fault through get controller_state. The automatic close command is documented as implemented in the current D2 update (VT-FAU-01 is the proposed verification case).

ID	D1 source	Specification
PS-FAU-02	FR-FAU-03 US-FAU-01	If the sensor driver reports a cabinet as faulted, or an open/close command fails, the ADC shall log the fault, expose the cabinet fault status through controller_state, and reject all new dispense/restock requests for that cabinet (REJECTED). If a motor open failure follows a provisional inventory change, PS-DIS-01/PS-INV-05 restores only the most recent delta; no failed opening may be reported as SUCCESS.
PS-OPE-01	FR-OPE-01 US-SEC-01	The ADC shall be in exactly one of seven states: Standby, Dispensing, Restocking, Maintenance, Locked, EmergencyShutdown, Fault. It begins in Standby subject to the startup safety check. All transitions shall conform to Table 2.3 and be logged with the initiating actor. Logout from Locked, Fault, or EmergencyShutdown ends the session but preserves that safety state; an authorized recovery action may leave Fault or EmergencyShutdown only after the relevant physical fault is cleared. D1 had four modes; D2 distinguishes Restocking and the additional safety/maintenance states.
PS-REL-01	NFR-REL-02, NFR-AVA-01 US-REL-01	Inventory, user-account and audit-log updates shall be persisted before the corresponding operation reports completion. On restart, prior inventory and user records are loaded and no user is logged in; the ADC may enter Standby only after an initial sensor/safety check confirms no active unsafe condition, otherwise it enters Fault. The controller serializes operations under one reentrant lock because the GUI, cabinet monitor, and serial listener may call concurrently. Persistence and thread resilience support NFR-REL-02 and NFR-AVA-01 but do not alone prove their quantitative reliability targets.
PS-USE-01	NFR-USE-01, NFR-USE-02 US-USE-01/02/03	Every routine operation (login, dispense, restock, close, logout) shall use only single-point touch input: no multi-touch, no touch-and-hold, no two-handed gesture. After login, a dispense shall take at most 4 touch interactions. Every error shall be shown with what the user should do next.

Table 2.2: What each user level may do (PS-AUT-03).

Level	May log in during	May switch the ADC into	Medicine	Other
Nurse	Standby	Dispensing, Restocking	Own allowed list only	Read inventory, close cabinets

Level	May log in during	May switch the ADC into	Medicine	Other
Maintenance	Standby, Maintenance, Fault, EmergencyShutdown	Maintenance	None	Read inventory
Admin	Any state	Standby, Maintenance, Locked, EmergencyShutdown, Fault	None	Add, edit and delete users; read inventory

Table 2.3: State changes (PS-OPE-01).

From	Event or condition	To
Standby	A logged-in Nurse requests Dispensing or Restocking	Dispensing or Restocking
Dispensing, Restocking	The nurse logs out (all open cabinets are closed first)	Standby
Dispensing, Restocking	The nurse switches between the two	Restocking, Dispensing
Standby	A logged-in Maintenance user or Admin requests Maintenance	Maintenance
Maintenance	The user logs out	Standby
Any state	A cabinet has been open for 180 s (PS-FAU-01)	Fault
Locked, Fault, EmergencyShutdown	The logged-in user logs out; session is cleared and any safe close action is attempted	Same state (safety state is preserved)
Any state	Logged-in Admin requests an allowed state; from Fault/EmergencyShutdown, recovery preconditions must be satisfied before Standby or Maintenance	Requested state only if its transition guard holds; otherwise NOT_ALLOWED_IN_STATE
Locked	A user logs out (Locked is kept, so a login and logout does not unlock)	Locked

From	Event or condition	To
Locked	An authenticated Admin authorizes unlock or recovery; Fault/EmergencyShutdown also require verified clearance of the unsafe condition	Standby only if recovery checks pass
Fault, Locked, EmergencyShutdown	A Nurse requests Dispensing or Restocking	Refused (NOT_ALLOWED_IN_STATE)

2.4 Preconditions, Postconditions, and Invariants

Notation: x' is the value of x after the operation. stock(c, m) is the quantity of medicine m in cabinet c and total(c) is the sum of stock(c, m) over all m.

Operation 1: login(username, password) and card login(username) [PS-AUT-01, PS-AUT-02]

Precondition	No user is logged in. Password login requires the user not be locked out (failed(u)<3 or 60 s elapsed). Card login requires a username assertion originating from the physical card reader after its own decryption and verification; untrusted inputs cannot use this path.
Postcondition (success)	On valid credentials (or a verified card assertion), and permitted role/state, create one session with username, level and allowed medicines; failed'(u)=0; return SUCCESS and persist a log entry.
Postcondition (failure)	No session exists. Unknown user: NOT_FOUND. Wrong password: PASSWORD_INCORRECT and failed'(u) = failed(u) + 1 with the time stored. Lockout: LOCKED_OUT and the stored count and time do not change. Level not allowed in this state: NOT_ALLOWED_IN_STATE.
Invariants	At most one session exists. No cabinet is opened without a session. 0 ≤ failed(u) ≤ 3.

Operation 2: dispense(cabinet c, medicine m, quantity q) [PS-DIS-01]

Precondition	A Nurse is logged in; state = Dispensing; 1 ≤ c ≤ N; m is on the nurse's allowed list; c is not in "fault"; q is an integer with 1 ≤ q ≤ stock(c, m).
Postcondition (success)	stock'(c,m)=stock(c,m)-q; motor reports OPEN within 5 s; matching audit entry exists; return SUCCESS. The operation is serialized with other inventory updates.

Postcondition (failure)	Return a PS-DIS-01 failure code without a successful dispense. If motor opening fails after the pending -q update, roll back that transaction's latest delta and restore stock'(c,m)=stock(c,m); preserve unrelated committed transactions. Log ERROR and the rollback outcome. On rollback failure, log an unresolved discrepancy, mark the drawer faulted, and never return SUCCESS.
Invariants	stock(c,m)>=0. Failed motor opening never results in SUCCESS. No unauthorized opening. A failed request may not leave behind an unreported committed inventory decrement.

Operation 3: restock(cabinet c, medicine m, quantity q) [PS-INV-02, PS-INV-03, PS-INV-05]

Precondition	A Nurse is logged in; state = Restocking; 1 ≤ c ≤ N; m is on the nurse's allowed list; c is not in "fault"; q is a positive integer; stock(c, m) + q ≤ 100; total(c) + q ≤ 300.
Postcondition (success)	stock'(c, m) = stock(c, m) + q; cabinet c is OPEN; a log line is written; the result is SUCCESS.
Postcondition (failure)	Reject on a violated limit without changing inventory. On open failure after the pending +q write, undo precisely that most recent delta so stock'(c,m)=stock(c,m), and return ERROR. Log failure and rollback; on rollback failure, mark a fault and prohibit SUCCESS.
Invariants	0 ≤ stock(c, m) ≤ 100 for every pair. 0 ≤ total(c) ≤ 300 for every cabinet.

Operation 4: checkOpenCabinets() [PS-FAU-01]

Precondition	Runs every 1 s. open_time(c) is the time since cabinet c was first seen OPEN.
Postcondition	At the first sample with open_time(c)>=60 s, record one WARNING; at the first sample with open_time(c)>=180 s, record one ERROR, issue CLOSE, record its return status, and set state'=Fault regardless of close success. On a new closed-to-open cycle, clear the previous timer.
Invariant	Given a 1 s monitor period and continuous operation, any drawer still OPEN at 181 s has caused an attempted close and Fault state. If the sensor cannot verify closure, Fault persists; physical closure is never assumed solely from the command.

Operation 5: handle request "get inventory" [PS-INV-04, PS-EXT-01, PS-EXT-02]

Precondition	A request line has arrived while the link is up, with the right device_id, type "get" and target "inventory".
Postcondition	Exactly one response is sent with status SUCCESS and one record (cabinet, medicine, quantity) for every pair in the filter, read from the stored inventory. The inventory is not changed. If the inventory cannot be read, the response status is ERROR.
Failure (link down)	The request is not processed and no response is sent. The ADC keeps working locally. A request that waits in the queue is processed in order when the link returns.
Invariant	Every request gets at most one response. A response is never sent for a request that was not carried out.

System-wide invariants

- **INV-1** A cabinet is opened only by a dispense or restock request of a logged-in Nurse in Dispensing or Restocking state.
- **INV-2 0 <= stock(c,m) <= 100 for each medicine in each cabinet.**
- **INV-3** 0 ≤ total(c) ≤ 300 for every cabinet.
- **INV-4** 0 ≤ failed(u) ≤ 3 for every user, and a user with failed(u) = 3 cannot log in by password for 60 s.
- **INV-5 In Fault, Locked and EmergencyShutdown, no new dispense or restock request is accepted; logout preserves the current safety state.**
- **INV-6** Exactly one state is active at any time, and at most one user is logged in.
- **INV-7** Every change to the inventory by a dispense or restock has a matching line in the event log.

INV-8 A failed motor opening cannot produce SUCCESS; the most recent transaction delta must be undone (or an unresolved inventory discrepancy and cabinet fault must be recorded).

INV-9 Fault and EmergencyShutdown may return to Standby only through an authorized recovery transition after the unsafe condition is cleared.

2.5 Coverage of D1 Requirements

D1 requirement	Precise specification(s)
FR-AUT-01	PS-AUT-01, PS-AUT-03
FR-INV-01	PS-INV-01, PS-INV-05
FR-INV-02	PS-INV-04, PS-EXT-01
FR-DIS-01	PS-DIS-01
FR-EVE-01	PS-EVE-01, PS-INV-01
FR-FAU-01	PS-AUT-02
FR-FAU-02	PS-FAU-01
FR-FAU-03	PS-FAU-02
FR-OPE-01	PS-OPE-01
FR-EXT-01	PS-EXT-01, PS-EXT-02
New FR-INV-03 (capacity limits)	PS-INV-02, PS-INV-03, PS-INV-05
NFR-SEC-01, NFR-SEC-02	PS-AUT-01, PS-AUT-02, PS-AUT-03
NFR-REL-01	PS-INV-04, PS-INV-06 (VT-INV-01)
NFR-REL-02, NFR-AVA-01	PS-REL-01
NFR-PER-01	PS-DIS-01
NFR-TRA-01	PS-EVE-01
NFR-USE-01, NFR-USE-02	PS-USE-01

The major additions and revisions to D1, with their rationale, are summarized in Table 1.1 of the Introduction. Specification PS-INV-06 makes the NFR-REL-01 accuracy measure verifiable and distinguishes software-ledger accuracy from physical stock verification.

3. Modular Design and Interfaces

Each module below is documented under the same headings: Responsibility, Secret, Exported operations, Semantics, Assumptions, Effects and Exceptions. In the tables, status means a result code from `status_codes.py`, and cabinet means a drawer number from 1 to 4 (a decimal string such as `"1"` is also accepted where the protocol sends IDs as strings). Unless stated otherwise, every operation logs its outcome through the Event Logger (3.3.1).

The decomposition follows Parnas information hiding: each module owns an anticipated change and exports operations that keep the hidden decision out of its clients. The matrix below makes those design secrets explicit; subsequent interface tables specify inputs, outputs, side effects, failure behaviour, and assumptions.

3.1 Hardware-Hiding Modules

3.1.1 Cabinet Motor and Sensor Driver

Responsibility: This is the only module that talks to the drawer hardware. It moves drawers and reports what the sensors detect. It is currently a **simulator** with the same interface the real firmware driver will have.

Secret: The motor command protocol, how sensors are polled, and how a drawer fault is detected. Replacing the simulator with real firmware changes only this module.

Exported operations:

Operation	Inputs	Outputs
`CabinetMotorAndSensorDriver(number_of_cabinets)`	int	driver object
`get_cabinets_states()`	none	list of `"open"`, `"closed"` or `"fault"`
`open_cabinet(cabinet)`	int 1..N	status
`close_cabinet(cabinet)`	int 1..N	status
`set_cabinets_states(states)`	list of state strings	none

Semantics:

- `CabinetMotorAndSensorDriver`: creates a driver for N drawers, all starting as `"closed"`.
- `get_cabinets_states`: returns the current sensor reading for every drawer, with index i mapping to cabinet i+1. In the documented prototype this is the internal mutable list; returning a defensive copy is a recommended interface-hardening change, not an implementation claim.
- `open_cabinet`: drives the drawer open. Returns `SUCCESS`, or `ERROR` if the motor fails (real hardware only, because the simulator always succeeds).
- `close_cabinet`: drives the drawer closed. Returns the same codes as `open_cabinet`.
- `set_cabinets_states`: **simulation and testing only.** Replaces the sensor readings, for example to inject a `"fault"`. It will be removed when real hardware is used.

Assumptions:

- The cabinet number is already valid (1 to N). The controller checks it before every call.
- Callers currently receive the driver's internal mutable list and must not modify it. The module contract should be strengthened to return a defensive copy in a later implementation; the test-only set_cabinets_states operation remains restricted to simulator/testing use and must not be exposed through the production controller or protocol APIs.
- `open_cabinet` and `close_cabinet` move the drawer and update the state the driver reports.
- Every operation is logged: open and close at INFO, an unexpected starting state (opening an open drawer, or acting on a faulted drawer) at WARNING, and sensor reads at DEBUG.

Exceptions:

- The driver does not check the cabinet number. A number above N raises `IndexError`. A number of 0 or less selects a drawer from the end of the list, so the wrong drawer moves.
- A motor failure is reported as the status `ERROR`, not as an exception.

3.1.2 Serial Driver

Responsibility: Sends and receives complete text messages over the physical link to the Monitoring System, buffers received messages, and reports the link status.

Secret: pyserial, framing (splitting the byte stream at `\n`), the RX thread and queue, and the retry and reconnect strategy. Switching to TCP would only need a new class with the same interface.

Exported operations:

Operation	Inputs	Outputs
`SerialDriver(port, baudrate, bytesize, parity, stopbits, timeout, write_timeout, xonxoff, rtscts, dsrdtr, line_ending, rx_queue_size)`	the `serial` settings from `config.yaml`	driver object
`open()`	none	none
`close()`	none	none
`get_status()`	none	`"Ready"`, `"Reconnecting"` or `"Off"`
`send(message)`	str	none
`read()`	none	str or `None`
`read_all()`	none	list of str
`add_rx_message(message)`	str	bool

Semantics:

- `SerialDriver`: creates the driver with status `"Off"` and an empty receive queue. The port is not opened yet.

- `open`: opens the port. Status becomes `"Ready"`, or `"Reconnecting"` while it retries after a failure.
- `close`: closes the port. Status becomes `"Off"`.
- `get_status`: returns the link status. `"Ready"` is the only status in which messages are exchanged.
- `send`: adds `line_ending` to the message and transmits it, blocking for at most `write_timeout` (1 s).
- `read`: returns the oldest complete received message without its terminator, or `None` if the queue is empty. Never blocks.
- `read_all`: returns every queued message in arrival order, or an empty list. Never blocks.
- `add_rx_message`: puts one complete received message on the queue. It is used by the RX listener, and by tests to inject messages without hardware. Returns `True` if queued, and `False` if the message was dropped.

Assumptions:

- `message` passed to `send` contains no literal newline, because the newline is the frame terminator. `json.dumps` already guarantees this.
- Callers check that `get_status()` is `"Ready"` before sending or reading.
- Only one module (3.2.4) uses the driver

Effects:

- `open` and `close` acquire and release the port and change the status.
- `send` writes bytes to the link.
- `read` and `read_all` remove the messages they return from the queue.
- `add_rx_message` adds a message to the queue, or drops it if the queue is full (`rx_queue_size` = 100).
- Open, close and send are logged at INFO. A dropped message is logged at WARNING. A rejected argument is logged at ERROR.

Exceptions:

- No operation raises.
- Problems are reported in other ways: `add_rx_message` returns `False` if the queue is full or the message is not a string, a lost link shows as status `"Reconnecting"`, and a failed send is logged at ERROR and the message dropped.

Implementation status: The hardware code for `open`, `send` and the RX listener is a stub that only logs. Received messages are injected with `add_rx_message`.

3.2 Behaviour-Hiding Modules

3.2.1 Cabinet Controller

Responsibility: The central ADC logic. It holds the cabinet state and the logged-in user, enforces every access rule, coordinates the inventory with the drawers, and monitors how long drawers stay open.

Secret:

- The state machine and the role tables `LOGIN_ALLOWED_IN`, `CAN_ENTER` and `CAN_ACCESS_MEDICATION`.
- The open-drawer time limits.
- The order of each medicine action: update the inventory, then open the drawer, and roll the inventory back if the drawer fails to open.

  Adding a user level or a state changes only this module.

Exported operations:

Operation	Inputs	Outputs
`Cabinet_Controller()`	none	controller object
`ID`, `state`	none (read-only attributes)	str
`username_password_login(username, password)`	str, str	status
`username_login(username)`	str	status
`logout()`	none	status
`change_state(state)`	state name	status
`dispense_medicine(cabinet, medicine, quantity)`	cabinet, str, int	status
`add_medicine(cabinet, medicine, quantity)`	cabinet, str, int	status
`close_cabinet(cabinet)`	cabinet	status
`check_inventory()`	none	`(status, inventory)`
`get_cabinet_states()`	none	`{cabinet: "open" / "closed" / "fault"}`
`add_user(username, password, level, allowed_medication=())`	str, str, str, iterable of str	status
`edit_user(username, **changes)`	str, field=value pairs	status
`delete_user(username)`	str	status
`load_users()`	none	dict or `None`

Semantics:

- `Cabinet_Controller`: creates and starts the whole ADC. It builds 3.2.2, 3.2.3 and 3.1.1, starts the drawer monitor thread, then starts 3.2.4. The initial state is `Standby`, nobody is logged in, and `ID` is `"000"`.
- `ID`: the device ID.
- `state`: the cabinet state, one of `Standby`, `Dispensing`, `Restocking`, `Maintenance`, `Locked`, `EmergencyShutdown` or `Fault`.
- `username_password_login`: logs in a user with a password.
  - Returns `USER_ALREADY_LOGGED_IN` if someone is already logged in. This is checked first, so it does not count as a failed attempt.
  - Otherwise returns the result of 3.2.2: `SUCCESS`, `PASSWORD_INCORRECT`, `NOT_FOUND` or `LOCKED_OUT`.

    - If the credentials are correct but the user's level may not log in during the current state, returns `NOT_ALLOWED_IN_STATE`.
    - On `SUCCESS` the user becomes the current user. The state does not change.
- `username_login`: receives only a verified username from the trusted physical card reader after the reader has performed decryption and authenticity verification. This operation is not exposed as arbitrary username-only login to the GUI or Monitoring System. Password lockout does not apply to a verified card event.
- `logout`: returns `NOT_LOGGED_IN` when no user is logged in. Otherwise it attempts to close open drawers and clears the session. Dispensing, Restocking and Maintenance return to Standby when safe; Locked, Fault and EmergencyShutdown retain their state until an explicitly authorized unlock/recovery action is accepted. Close failure leaves the system faulted rather than silently entering Standby.
- `change_state`: changes state only at a logged-in user's authorized request, subject to role/transition guards. Nurse may request Dispensing or Restocking; Maintenance may request Maintenance; Admin may request Standby, Maintenance, Locked, EmergencyShutdown or Fault. From Fault/EmergencyShutdown, Standby or Maintenance additionally requires verification that unsafe conditions are cleared. Returns SUCCESS, NOT_LOGGED_IN, INVALID_VALUE, ACCESS_DENIED or NOT_ALLOWED_IN_STATE as applicable.
    - Nurse: `Dispensing`, `Restocking`.
    - Maintenance: `Maintenance`.
    - Admin: `Standby`, `Maintenance`, `Locked`, `EmergencyShutdown`, `Fault`.
      Returns `SUCCESS`, `NOT_LOGGED_IN`, `INVALID_VALUE` (unknown state), `ACCESS_DENIED` (the level may not enter that state), or `NOT_ALLOWED_IN_STATE` (`Dispensing` or `Restocking` requested while the state is `Fault`, `Locked` or `EmergencyShutdown`).
- `dispense_medicine`: dispenses medicine to a Nurse. Requires a Nurse logged in, the state `Dispensing`, the medicine in the Nurse's allowed list, and a drawer that is not in fault.
    - Returns `SUCCESS`, `NOT_LOGGED_IN`, `ACCESS_DENIED`, `NOT_ALLOWED_IN_STATE`, `INVALID_VALUE` (bad cabinet, name or quantity), `NOT_FOUND` (the medicine is not in that cabinet), `REJECTED` (not enough stock, or drawer in fault), or `ERROR` (file error, or the drawer failed to open).
    - The drawer stays open until `close_cabinet` or `logout` is called.
- `add_medicine`: same as `dispense_medicine`, for restocking. Requires the state `Restocking`. `REJECTED` means a stock limit would be exceeded or the drawer is in fault.
- `close_cabinet`: closes one drawer for the logged-in user. Returns `NOT_LOGGED_IN`, `INVALID_VALUE`, or the status from 3.1.1.
- `check_inventory`: lists the stock for any logged-in user, in any state. On `SUCCESS`, `inventory` is `{medicine: {"total": int, "cabinets": {cabinet: qty}}}`, sorted by medicine, with zero quantities left out. On `NOT_LOGGED_IN` or `ERROR`, `inventory` is `None`.
- `get_cabinet_states`: returns the current sensor state of each drawer.
- `add_user`, `edit_user`, `delete_user`: the same as 3.2.2, plus these rules:
    - `add_user` rejects an unknown level with `INVALID_VALUE`.
    - Deleting the logged-in user logs them out.
    - Editing the logged-in user's level or allowed medicines takes effect immediately.

- `load_users`: the same as 3.2.2 (no passwords), but returns `None` instead of raising on a file error.

Assumptions:

- Only one ADC controller runs per process, and only one user is logged in at a time.
- Callers (3.2.4 and the GUI) have checked the types of the arguments. The controller checks their values.
- Callers do not set `ID` or `state` directly. They only read them.
- The drawer sensors reflect the real drawer positions. The monitor timers are based on them.

Effects:

- Every public operation holds one reentrant lock, so the GUI, communication and monitor threads never see partly updated state.
- Login and logout change the current user. Through 3.2.2, login also writes the failed-attempt counter and the time of the last attempt to the user file.
- `logout` and `change_state` change `state`. `logout` also closes drawers.
- `dispense_medicine` and `add_medicine` write the inventory file through 3.2.3 and open a drawer through 3.1.1.
- User operations write the user file through 3.2.2.
- Every outcome is logged, including the reason for each rejection and the user who made the request.
- The drawer monitor runs on its own thread every 1 s. At 60 s open it logs WARNING once; at the first check at/after 180 s, it issues close_cabinet, logs its result and enters Fault even if closure fails. The auto-close call is marked implemented in this D2 documentation update. The timer resets after a closed-to-open cycle.
- The constructor passes on any exception raised by the modules it builds (see 3.2.2, 3.2.3 and `config.yaml`).
- `username_password_login` and `username_login` pass on `OSError`, `KeyError` or `ValueError` from 3.2.2 if the user file cannot be read or is malformed.
- Every other operation reports failure as a status code, or as `None` for `load_users`.
- If the drawer fails to open, precisely that request's most recent inventory change is rolled back before ERROR is returned. If rollback persistence fails, an unresolved stock discrepancy is logged and the drawer remains faulted.
- The monitor thread catches and logs every exception, and keeps running.

3.2.2 Authenticator

Responsibility: Stores user accounts and checks credentials, including the lockout after failed attempts. It does not decide whether a user's level may log in during the current state.

Secret:

- The CSV layout of the user file, and the `;` separator for medicine lists.
- How passwords are stored and compared (plaintext in this prototype; salted hashing required before clinical deployment). This is an explicit prototype security limitation, not a production recommendation.

- The lockout policy: 3 consecutive failed attempts lock the user out for 60 s.

  Moving to hashed passwords or a database changes only this module.

Exported operations:

Operation	Inputs	Outputs
`Authenticator()`	none	authenticator object
`check_username_password(username, password)`	str, str	`(status, level, allowed_medication)`
`check_username(username)`	str	`(status, level, allowed_medication)`
`add_user(username, password, level, allowed_medication=())`	str, str, str, iterable of str	status
`edit_user(username, **changes)`	str, field=value pairs	status
`delete_user(username)`	str	status
`load_users()`	none	`{username: {level, last_login_attempt, consecutive_failed_attempts, allowed_medication}}`

Semantics:

- `Authenticator`: opens the user file named in `config.yaml`, and creates it with a header row if it is missing.
- `check_username_password`: checks a username and password.
  - `SUCCESS`: the password is correct. Returns the user's level and the list of allowed medicine names.
  - `PASSWORD_INCORRECT`: the password is wrong.
  - `LOCKED_OUT`: the user has failed 3 or more times in a row, and the last attempt was less than 60 s ago. The password is not checked. After 60 s the user gets a fresh set of attempts.
  - `NOT_FOUND`: no such user.
  - On every result except `SUCCESS`, `level` and `allowed_medication` are `None`.
- `check_username: resolves the verified username supplied by a trusted, self-authenticating hardware reader; the driver must first complete all card decryption and verification. It does not authenticate an untrusted typed username. Ignores password lockout only for this trusted input and returns SUCCESS or NOT_FOUND.`
- `add_user`: adds a new user with no failed attempts. Returns `SUCCESS`, `ALREADY_EXISTS`, or `ERROR`.
- `edit_user`: changes only the fields given. `username` cannot be changed. Sending `consecutive_failed_attempts=0` clears a lockout. Returns `SUCCESS`, `NOT_FOUND`, `INVALID_VALUE` (unknown or forbidden field), or `ERROR`.
- `delete_user`: removes a user. Returns `SUCCESS`, `NOT_FOUND`, or `ERROR`.

- `load_users`: returns every user record. **Passwords are never included!**

Assumptions:

- Only one Authenticator writes to the user file at a time. The controller's lock guarantees this.
- The user file is only changed through this module.
- Usernames contain no commas or newlines, and medicine names contain no `;`.
- `level` values have already been checked by the controller.

Effects:

- The check operations update `consecutive_failed_attempts` and `last_login_attempt` in the user file. A successful check resets the counter to 0. A wrong password increments it. A `LOCKED_OUT` or `NOT_FOUND` result writes nothing.
- `add_user`, `edit_user` and `delete_user` rewrite the user file.
- Every attempt and every change is logged. Successes are logged at INFO, rejections at WARNING and file errors at ERROR. Passwords are never logged.

Exceptions:

- The constructor raises `OSError` or a `yaml` error if the configuration file or the user directory is unavailable.
- `check_username_password`, `check_username` and `load_users` raise `OSError` if the user file cannot be read. They raise `KeyError` or `ValueError` if the file is malformed.
- `add_user`, `edit_user` and `delete_user` catch file errors and return `ERROR` instead.

3.2.3 Inventory Tracker

Responsibility: Stores how much of each medicine is in each cabinet and enforces the stock limits. Each change is all or nothing.

Secret:

- The file format of the inventory.
- The atomic-write technique.
- Where the stock limits come from (`config.yaml`).
- How medicine names are normalised.

Exported operations:

Operation	Inputs	Outputs
`InventoryTracker(number_of_cabinets)`	positive int	tracker object
`get_inventory(cabinet=None, medicine=None)`	optional cabinet, optional str	list of `{"cabinet": int, "medicine": str, "quantity": int}`, or `None`
`add_medicine(cabinet, medicine, quantity)`	cabinet, str, int > 0	status
`dispense_medicine(cabinet, medicine, quantity)`	cabinet, str, int > 0	status
`reset_medicine()`	none	status

Semantics:

- `InventoryTracker`: opens the inventory file named in `config.yaml`, and creates it if it is missing.
- `get_inventory`: returns every entry that matches the filters. With no filters it returns every entry. Returns `None` if a filter is invalid, or if the file cannot be read or is malformed.
- `add_medicine`: adds stock to a cabinet. Returns `SUCCESS`, `INVALID_VALUE` (bad cabinet, name or quantity), `REJECTED` (the result would exceed `max_medicine_quantity` or `max_cabinet_quantity`), or `ERROR`.
- `dispense_medicine`: removes stock from a cabinet. Returns `SUCCESS`, `INVALID_VALUE`, `NOT_FOUND` (the medicine is not in that cabinet), `REJECTED` (not enough stock), or `ERROR`. **It never dispenses part of a request.**
- `reset_medicine`: sets every quantity to 0 and keeps the entries. Returns `SUCCESS` or `ERROR`.
- Medicine names are compared without regard to case or surrounding spaces, and are returned in lowercase.

Assumptions:

- The inventory file is only changed through this module.
- `number_of_cabinets` is not lowered while existing entries use the higher cabinet numbers. If it is, the file is reported as malformed.
- Permission checks (user level, state, allowed medicines) were already done by the controller.

Effects:

- `add_medicine`, `dispense_medicine` and `reset_medicine` rewrite the inventory file. A rejected request leaves the file unchanged.
- Each read, change and write cycle holds an internal lock, so concurrent calls cannot overwrite each other's changes.
- The file is written to a temporary file and then renamed. A crash during a write therefore never leaves a half-written inventory.
- Every change is logged at INFO, rejections at WARNING, and invalid input and file errors at ERROR.

Exceptions:

- The constructor raises `ValueError` if `number_of_cabinets` is not a positive integer. It raises `OSError` or a `yaml` error if the configuration or the data directory is unavailable.
- Other operations do not raise. A file error, a malformed file or bad input is reported as `ERROR`, `INVALID_VALUE`, or `None`.

3.2.4 External Communication Driver

Responsibility: Implements the ADC side of the Monitoring System protocol. It decodes each request, checks the device ID, calls the matching 3.2.1 operation, and sends exactly one response per request.

Secret: JSON, the field names, and how each (`type`, `target`) pair maps to a controller operation. A protocol revision changes only this module and the protocol document.

Exported operations:

Operation	Inputs	Outputs
`ExternalCommunicationDriver(cabinet_controller)`	the 3.2.1 instance	driver object
`start()`	none	none
`stop()`	none	none
`is_running()`	none	bool
Protocol requests over the link	JSON request (see communication_protocol.md)	one JSON response

Semantics:

- `ExternalCommunicationDriver`: stores the controller and reads `poll_interval` from `config.yaml`. It does not open the link.
- `start`: opens 3.1.2 and starts a listener thread that handles received messages every `poll_interval` (50 ms). If it is already running, it does nothing.
- `stop`: stops the listener thread, waits for it to end, and closes the link.
- `is_running`: returns `True` from `start` until `stop`.

Request handling: each request is checked in this order:

1. It is a valid JSON object.
2. The fields `device_id`, `type` and `target` are present.
3. `params` is an object.
4. `type` is `get` or `set`.
5. `device_id` is this controller's `ID`, or `"*"` together with `get controller_state`.
6. The target is known.

   At the first failed check, it answers with `INVALID_REQUEST`, `WRONG_DEVICE` or `UNKNOWN_TARGET`. Otherwise it checks the parameter types, calls the controller, and copies the controller's status into the response unchanged. Error responses may include `data.errorMessage`, a human-readable explanation.

Assumptions:

- The controller is fully built before `start` is called, because the listener can call it straight away.
- The Monitoring System follows the rules in the communication protocol.
- The controller checks the values in a request. This module only checks their types.

Effects:

- `start` and `stop` open and close the link, and start and stop the listener thread.
- Each request causes exactly one response on the link, and possibly one controller call, with that call's effects.
- A request that fails a check changes nothing on the ADC.

- Start and stop are logged at INFO, a repeated `start` at WARNING, and handler failures at ERROR with a stack trace.

Exceptions:

- The constructor raises `OSError` or a `yaml` error if the configuration file is unavailable. `start` raises `KeyError` if the `serial` section is incomplete.
- In the listener, an unexpected exception while handling one message is logged and answered with `ERROR`. The thread keeps running. If even the error response cannot be sent, that is logged too.
- Problems in a request are never raised. They are reported to the Monitoring System as status codes.

Implementation status: `get auth_details` and the `add` action of `set auth_details` are implemented. The other targets in the protocol document are specified but not yet implemented.

3.3 Common Services

3.3.1 Event Logger

Responsibility: Records every significant ADC event in one persistent log: logins, state changes, dispensing, drawer actions and communication errors.

Secret: Where and how events are recorded: file location, line format, console output, and future rotation or remote logging.

Exported operations:

Operation	Inputs	Outputs
`EventLogger()`	none	logger wrapper
`.logger.debug / info / warning / error / exception(msg)`	str	none
`clr_logs()`	none	`0` on success, `1` on failure

Semantics:

- `EventLogger`: gives access to the one shared ADC log.
- `.logger.<level>(msg)`: adds a timestamped entry at that level:
  - **DEBUG:** routine events, such as sensor reads and communication traffic.
  - **INFO:** normal actions, such as a drawer opening or closing, or a login.
  - **WARNING:** rejected requests, and unexpected cases that were handled.
  - **ERROR:** system failures.
    `exception` logs at ERROR and adds the current stack trace.
- `clr_logs`: empties the log file.

Assumptions:

- The caller of `clr_logs` has already checked that the user is an Admin.

- Messages never contain passwords.

Effects:

- The first `EventLogger()` in a process creates the log directory and attaches a file handler (DEBUG and above) and a console handler (INFO and above). Later instances reuse the same handlers, so every module writes to the same log and no entry is written twice.
- Each logging call appends a line to the log file, and also prints it to the console if its level is INFO or above.
- `clr_logs` deletes every existing entry.

Exceptions:

- The constructor raises `OSError` or a `yaml` error if the configuration file is missing or unreadable.
- Logging calls do not raise. Errors writing the log are handled by Python's `logging` package.
- `clr_logs` returns `1` instead of raising.

3.4 External Interface to the Monitoring System

This interface is documented from the ADC's side. The full message specification is in `communication_protocol.md`, including a JSON example for every request. This section summarises that specification and refers to its sections as §1 to §8.

Provides

The ADC answers requests. It never sends a message on its own: every message it sends is the one response to one request .

Message format:

- Every message is one UTF-8 JSON object on one line, ending in `\n`.
- Every response contains these fields:
  - `device_id`: the ADC's own ID.
  - `type`: always `"response"`.
  - `target`: copied from the request.
  - `status` and `status_name`: the result code and its name.
- A successful `get` response also contains `data`. A `set` response also contains `field`.

Direction	Message	Meaning	Timing
Monitor → ADC	`get controller_state`	Reads the ADC's ID and current state. It is the only request that may be sent to `device_id: "*"`, so it can find an ADC whose ID is not yet known (§5.1).	On demand
Monitor → ADC	`get auth_details(username?)`	Reads one user record, or all of them. Passwords are never sent (§5.2).	On demand

Monitor → ADC	`get inventory(cabinet?, medicine?)`	Reads stock levels, for example to detect low stock (§5.3).	On demand
Monitor → ADC	`get event_logs(min_level, since?, max_entries?)`	Reads logged events at or above a severity level, newest first. With `since`, it returns only events after that time (§5.4).	On demand
Monitor → ADC	`set device_id(value)`	Gives the ADC a new ID. The response already carries the new ID (§6.1).	On demand
Monitor → ADC	`set auth_details(action, username, ...)`	Adds, edits or deletes a user. Setting `consecutive_failed_attempts` to 0 with an edit clears a lockout (§6.2).	On demand
Monitor → ADC	`set state(value)`	Requests Standby, Maintenance, Locked or EmergencyShutdown. Unlock/recovery from Fault or EmergencyShutdown is allowed only after an authorized recovery request and confirmation that the unsafe condition has cleared (§6.3).	On demand
ADC → Monitor	`response(device_id, target, status, status_name, data or field)`	The result of exactly one request. `status` is one of the codes 0 to 8 in §4.	Within 1 s of the request (§8). Never sent unprompted.

Timing:

Nominal end-to-end response requirement: <= 1 s from complete request reception to response transmission while the link is Ready and one request is outstanding. The Monitoring System uses a 1 s response timeout and distinguishes a transport timeout from an ADC application-level status.

- The ADC checks for new requests every 50 ms (`poll_interval`) and handles them in the order they arrive. A response is normally sent within the 1 s response limit under nominal link conditions.
- Each response is written with a 1 s write timeout.

Meaning of the results :

- A message that cannot be parsed gets `INVALID_REQUEST` with `target: null`.
- An unknown target gets `UNKNOWN_TARGET`.
- A request addressed to another ADC gets `WRONG_DEVICE`, and the response carries the ADC's real ID.

- A valid request that the current state does not allow gets `REJECTED`. For example, `set state` to `Standby` or `Maintenance` is rejected while a user is logged in.
- `set state` to `Locked` or `EmergencyShutdown` always takes effect, and logs out the current user.

Implementation status: `get auth_details` and the `add` action of `set auth_details` are implemented. The other messages are specified in `communication_protocol.md` and are still being implemented.

Requires

- **Initiation:** The Monitoring System starts every exchange. The ADC never pushes a status or an alarm. The Monitoring System therefore learns about faults, low stock and lockouts by polling `get controller_state`, `get inventory`, and `get event_logs` with `since`. How quickly it sees an ADC fault depends on how often it polls.
- **Rate:** No more than one request may be outstanding per ADC. The Monitoring System waits for the response, or for the 1 s timeout, before it sends the next request to the same ADC. The ADC buffers up to 100 received messages and drops any more than that.
- **Ordering:** The ADC handles requests in the order they arrive. Responses carry no request ID. Because only one request is outstanding at a time, the next message on the line is always the response to the last request.
- **Acknowledgement:** Every request is acknowledged by exactly one response, and its `status` says whether the request succeeded. The ADC does not expect the Monitoring System to acknowledge its responses. A request with no response within 1 s is treated as timed out.
- **Retries:** A `get` may always be resent. A `set` that timed out may already have been applied, so the Monitoring System first reads the value with a `get`, and resends only if the change did not happen.
- **Addressing:** Each ADC has its own serial line. The Monitoring System gives every ADC a unique `device_id`, because the ADC cannot check that its ID is unique.
- **Link settings:** 115200 baud, 8 data bits, no parity, 1 stop bit, no flow control (`config.yaml`).

Protocol Secrecy

Only two modules know how the ADC communicates with the Monitoring System:

- **External Communication Driver (3.2.4)** is the only module that knows the message format: JSON, the field names, and how each request is routed to a controller operation.
- **Serial Driver (3.1.2)** is the only module that knows the physical link: the port, the framing, the receive buffer and reconnection.

Every other module works through Cabinet Controller (3.2.1) operations and never sees a message. As a result:

- A change to the message format changes only 3.2.4 and `communication_protocol.md`. Examples are moving from JSON to a binary format, renaming a field, or adding a target.
- A change to the link changes only 3.1.2. An example is replacing serial with TCP.
- **One deliberate exception:** the protocol status codes 0 to 8 are the same numbers the internal modules return (`status_codes.py`). This lets 3.2.4 forward a result without

translating it. If the protocol's codes ever had to change, 3.2.4 would translate them, and still no other module would change.

Degraded behaviour

The safety function never depends on the link. All of these are enforced inside the ADC using local data:

- access control
- the password lockout
- each user's list of allowed medicines
- the stock limits
- the open-drawer fault

The link is used only for supervision and remote administration.

Failure	ADC side	Monitoring System side
Link down	Keeps operating. Login, dispensing, restocking, lockout and the open-drawer fault all still work. Events are still written to the local event log. The Serial Driver reports `Reconnecting` and retries, and the listener pauses until the link is `Ready` again.	Requests time out after 1 s. After repeated timeouts, it marks the ADC as "communication lost" and keeps the ADC's last known state, with the time it was read.
Link restored	The listener resumes and answers requests. The ADC does not push the events it missed sending, because they are already kept in the event log.	Requests `get event_logs` with `since` set to its last successful poll, and adds the missed events to its history. Then it re-reads `controller_state` and `inventory` to clear out-of-date flags.
Partner never seen	Operates standalone with the default ID `000`. No ADC function waits for the Monitoring System. A silent link is normal, because the ADC never starts an exchange.	Finds the ADC by sending `get controller_state` to `"*"`. Until the ADC answers, shows it as absent, not as healthy.
Duplicate message	Messages have no sequence numbers. A duplicated `get` is harmless. A duplicated `set` may be applied twice. For some requests the second response shows this, for example `add` returns `ALREADY_EXISTS`.	Before resending a `set`, checks with a `get` whether the first one took effect.
Out-of-order message	Handles messages strictly in arrival order and answers each one once. With only one request outstanding at a time, the order cannot be mixed up.	Waits for each response or timeout before sending the next request. A response that arrives after its request timed out could be mistaken for the next request's

		response, so it checks that `target` matches the request.
Corrupted or partial message	Answers `INVALID_REQUEST` with `target: null`. Nothing is changed.	Treats the request as failed and resends it, following the rule for `set` above.
Wrong ADC on the line	Answers `WRONG_DEVICE` with its real ID. Nothing is changed.	Updates its record of which ADC is on that line.
Requests arrive too fast	Queues up to 100 messages. Any more are dropped, and each drop is logged.	Sees timeouts. Sending only one request at a time per ADC prevents this.

Implementation status: The reconnection, the logging of failed sends, and the receive listener belong to the Serial Driver (3.1.2), which is still a stub. The External Communication Driver (3.2.4) behaviour in the table is implemented: pausing while the link is not `Ready`, rejecting malformed messages, answering `WRONG_DEVICE`, and dropping messages when the queue is full.

4. Implementation Notes

The ADC control software is written in Python 3.10 or later, because it uses `match` statements. It depends on PyYAML and pyserial. The code is organised as follows:

- Each module is one file in `src/modules/`.
- Deployment settings are in `src/config/config.yaml`.
- Data the program keeps between runs is in `src/files/`, which is created on the first run. That data is the user accounts, the inventory and the event log.

Architecture diagram to code

The table maps each component of the Deliverable 1 high-level architecture diagram to the module and class that implements it.

D1 component	Module	Code	Notes
Cabinet Controller	3.2.1	`cabinet_controller.py`, class `Cabinet_Controller`	The central hub, as in D1. Its constructor creates the Authenticator, the Inventory Tracker, the drawer driver and the External Communication Driver. This implements the composition links in the D1 UML domain diagram.
Authenticator	3.2.2	`authenticator.py`, class `Authenticator`	"Auth results" in D1 is the `(status, level, allowed_medication)`

			tuple returned by `check_username_password` and `check_username`.
Authentication Devices (card)	3.2.1, 3.2.2	`Cabinet_Controller.username_login` → `Authenticator.check_username`	Card login is supported by the controller interface, while the physical reader driver is planned. The physical reader is assumed to decrypt and cryptographically verify cards before providing a verified username; an arbitrary username must not enter through this path.
Inventory Tracker	3.2.3	`inventory_tracker.py`, class `InventoryTracker`	"Changes" in D1 are `add_medicine` and `dispense_medicine`. "Inventory status" is `get_inventory`.
Event Logger	3.3.1	`event_logger.py`, class `EventLogger`	In D1 only the controller sends events. In the code, every module writes directly to one shared log, so each event is recorded where it happens, with its details.
Cabinet open/close motor driver, Cabinet open/close sensors	3.1.1	`cabinet_motor_and_sensor_driver.py`, class `CabinetMotorAndSensorDriver`	The two D1 components are merged into one module, which currently simulates the hardware. "Motor command" is `open_cabinet` and `close_cabinet`. "Cabinet drawer status" is `get_cabinets_states`.
External Communication Driver	3.2.4, 3.1.2	`external_communication_driver.py`, class `ExternalCommunicationDriver`; `serial_driver.py`, class `SerialDriver`	Split into two modules, so the message format and the physical link are separate secrets. It runs on its own thread.
Graphic User Interface	not yet implemented	`src/GUI/main_GUI.py` (empty)	The GUI is not implemented; controller methods are called

			directly for the demonstration. Any future GUI must enforce the trusted-reader boundary and one-handed-use constraints.
(not in D1)	shared	`status_codes.py`, `config.yaml`	Added in this deliverable: one set of result codes for every module, and the site settings, kept out of the code.

One change to a D1 data flow: In D1, "Acceptable auth details" goes from the External Communication Driver straight to the Authenticator. In the code, the request goes through the controller (`add_user`, `edit_user`, `delete_user`, `load_users`). The access rules are therefore enforced in one place, for example rejecting an unknown user level, and logging out a user who is deleted.

State machine

The D1 state diagram is implemented in `Cabinet_Controller`:

- **States:** the seven states are the strings in `ALL_STATES`. The current state is `Cabinet_Controller.state`. Every transition goes through `_set_state`, which logs "state changed from X to Y by <user>", so every transition in the demonstration appears in the log.
- **Dispensing and Restocking:** D1 uses one "Dispensing State" for both dispensing and restocking requests. The code splits it into `Dispensing` and `Restocking`, because it has to know which of `dispense_medicine` or `add_medicine` is allowed.
- **Guards** are tables, not conditions scattered through the code:
  - `LOGIN_ALLOWED_IN`: the states in which each user level may log in. A Nurse may log in only in `Standby`. Maintenance may log in in `Standby`, `Maintenance`, `Fault` and `EmergencyShutdown`. An Admin may log in in any state.
  - `CAN_ENTER`: the states a logged-in user of each level may request.
  - `NO_MEDICATION_ACCESS_STATES`: the states from which `Dispensing` and `Restocking` cannot be entered.
- **Concurrency:** every public method holds one lock, through the `_locked` decorator. A transition requested by the GUI, the communication thread or the drawer monitor thread therefore never interleaves with another.

D1 transition	Trigger in code	Guard or note
Standby → Dispensing (request with valid login)	`username_password_login` or `username_login`, then `change_state("Dispensing")` or `change_state("Restocking")`	Nurse level, valid credentials, not locked out. Checked against `LOGIN_ALLOWED_IN` and `CAN_ENTER`.

Dispensing → Standby (cabinet closed)	`logout()`	`logout()` first closes every drawer that is still open. `close_cabinet` on its own keeps the state, so a nurse can get several medicines in one session.
Dispensing → Fault (cabinet left open too long)	The drawer monitor thread runs `_check_open_cabinets` every 1 s.	The monitor issues an automatic CLOSE at the 180 s threshold (checked every 1 s), logs the outcome and enters Fault. Automatic close is documented as implemented; physical closure still depends on the motor/sensor result.
Standby → Fault (sensor reports unintended state, system errors)	Partly implemented.	A drawer whose sensor reports `fault` cannot be used for dispensing or restocking (`REJECTED`), but this does not yet move the cabinet to `Fault`.
Standby → Maintenance (maintenance request with valid login)	Maintenance login, then `change_state("Maintenance")`	Maintenance or Admin level.
Maintenance → Standby (maintenance log out)	`logout()`	None.
Standby → Locked (request from management system)	`set state` with value `Locked` (protocol §6.3), or an Admin's `change_state("Locked")`	Always takes effect. `logout()` keeps the state `Locked`, so logging in and out does not unlock the cabinet.
Locked → Standby (unlock request from management system)	`set state` with value `Standby`, or an Admin's `change_state("Standby")`	None.
Standby → Emergency Shutdown (request from management system)	`set state` with value `EmergencyShutdown`, or an Admin's `change_state("EmergencyShutdown")`	Always takes effect.

Fault → Emergency Shutdown (safety-critical errors)	An Admin's `change_state("EmergencyShutdown")`, or the management system	The ADC does not yet escalate to `EmergencyShutdown` automatically.
Fault → Standby, Emergency Shutdown → Standby (faults cleared)	`set state` with value `Standby`, or an Admin's `change_state("Standby")`	Explicit Admin or authorized monitoring recovery only, after verifying the relevant fault or unsafe condition has cleared. Logout alone never clears Fault/EmergencyShutdown.

Status of the management-system path: the `set state` request is specified in the protocol, and its handler in the External Communication Driver (3.2.4) is still being implemented. The Admin path through `change_state` works now.

Activity flow

The D1 activity diagram for a nurse getting medicine maps to controller calls as follows.

D1 activity	Code
Select intention via UI	Done by the GUI (planned). The intention becomes the state requested after login: `change_state("Dispensing")` or `change_state("Restocking")`.
Select log in method	Two controller methods: `username_password_login(username, password)` and `username_login(username)` for a card.
Log in using username/password	`Authenticator.check_username_password` checks the password and the lockout.
Card log in	`Authenticator.check_username is used only for a username that a trusted physical card reader has decrypted and verified. The physical reader integration is planned; the lower-level interface is present.`
Merge, then the failed login / successful login decision	Both login methods finish in `_complete_login`. A failed login returns a status (`PASSWORD_INCORRECT`, `NOT_FOUND`, `LOCKED_OUT` or `NOT_ALLOWED_IN_STATE`) and changes nothing, so the user goes back to choosing a login method. After 3 wrong passwords the user is locked out for 60 s.
Select drawer to open, Get medicine	`dispense_medicine checks Nurse, state, drawer, medicine permissions and sensor. It then stores the specific pending -q delta and commands OPEN. On failed opening, exactly that latest delta is rolled back before ERROR; other committed inventory is unaffected.`
Close the drawer	`close_cabinet(cabinet)`
More medicines to get (loop)	The state stays `Dispensing`, so the nurse calls `dispense_medicine` and `close_cabinet` again.
Done, Close the cabinet, End	`logout()` closes any drawer still open, returns the cabinet to `Standby` and clears the logged-in user.

Restocking follows the same flow, using `add_medicine` and the state `Restocking`.

Use cases to code

D1 use case	Code
Dispensing	`Cabinet_Controller.dispense_medicine`
Restocking	`Cabinet_Controller.add_medicine`
Log In/Authentication	`username_password_login` and `username_login`, which use the Authenticator (3.2.2)
Cabinet Open, Cabinet Close	`CabinetMotorAndSensorDriver.open_cabinet` and `close_cabinet`, called by the controller's `dispense_medicine`, `add_medicine`, `close_cabinet` and `logout`
Do Maintenance	Maintenance login, then `change_state("Maintenance")`
Access system logs, Access fault flags and internal data	The event log `src/files/events.log`, `get_cabinet_states` and `check_inventory`
Report System Status, Report System Logs, Report Inventory	`get controller_state`, `get event_logs` and `get inventory`, handled by the External Communication Driver (3.2.4). These are specified in the protocol and still being implemented.

5. Traceability Table

The table extends the D1 traceability table forward: each D1 requirement ID is traced to its precise specification (Section 2), to the module that implements it, and to the file, class and function in the code. Paths are inside src/. Module and class names are those used in the Modular Design section.

Module to code map

Module	File	Class	Role
Cabinet Controller	modules/cabinet_controller.py	Cabinet_Controller	Owns the state, the session and the rules; calls all other modules
Authenticator	modules/authenticator.py	Authenticator	Checks credentials, keeps failed-attempt counts, stores users
Inventory Tracker	modules/inventory_tracker.py	InventoryTracker	Stores and changes medicine quantities, enforces the limits

Module	File	Class	Role
Cabinet Motor and Sensor Driver	modules/cabinet_motor_and_sensor_driver.py	CabinetMotorAndSensorDriver	Opens and closes cabinets, reports their state
External Communication Driver	modules/external_communication_driver.py	ExternalCommunicationDriver	Reads Monitoring System requests, calls the controller, sends responses
Serial Driver	modules/serial_driver.py	SerialDriver	Sends and receives lines on the serial port
Event Logger	modules/event_logger.py	EventLogger	Writes the event log
Status Codes	modules/status_codes.py	(constants)	Result codes shared by all modules
GUI	GUI/main_GUI.py	[planned]	Touchscreen interface
Configuration	config/config.yaml	(data)	Limits, file paths and serial settings

Requirement to Code

Req ID	Requirement (short)	Precise spec	Module	Code (file: class.function)
FR-AUT-01	Staff must authenticate (password or card) before access	PS-AUT-01, PS-AUT-03	Authenticator, Cabinet Controller	authenticator.py: Authenticator.check_username_password, check_username cabinet_controller.py: Cabinet_Controller.username_password_login, username_login, _complete_login, LOGIN_ALLOWED_IN

Req ID	Requirement (short)	Precise spec	Module	Code (file: class.function)
FR-INV-01	Log each medicine with name, quantity and location	PS-INV-01, PS-INV-05	Inventory Tracker, Cabinet Controller	inventory_tracker.py: InventoryTracker.get_inventory, add_medicine, _read_rows, _write_rows cabinet_controller.py: Cabinet_Controller.check_inventory, add_medicine files/inventory.csv
FR-INV-02	Restocking need is known to the Monitoring System	PS-INV-04, PS-EXT-01	External Communication Driver, Inventory Tracker	inventory_tracker.py: InventoryTracker.get_inventory cabinet_controller.py: Cabinet_Controller.check_inventory external_communication_driver.py: ExternalCommunicationDriver._handle_message (target "inventory") [planned]
FR-DIS-01	Open the cabinet holding the medicine for the authenticated user	PS-DIS-01	Cabinet Controller, Inventory Tracker, Motor and Sensor Driver	cabinet_controller.py: Cabinet_Controller.dispense_medicine, _check_medication_request, close_cabinet inventory_tracker.py: InventoryTracker.dispense_medicine cabinet_motor_and_sensor_driver.py: CabinetMotorAndSensorDriver.open_cabinet, close_cabinet; most-recent transaction rollback on failed open in Cabinet_Controller.dispense_medicine
FR-EVE-01	Record dispensing, authentication, access and restocking events	PS-EVE-01	Event Logger	event_logger.py: EventLogger.logger (used as self.log in every module) files/events.log

Req ID	Requirement (short)	Precise spec	Module	Code (file: class.function)
FR-FAU-01	Detect 3 failed logins and notify	PS-AUT-02	Authenticator, External Communication Driver	authenticator.py: Authenticator.check_username_password (ALLOWED_CONSECUTIVE_FAILED_LOGINS, LOCKOUT_PERIOD), load_users external_communication_driver.py: ExternalCommunicationDriver._get_auth_details (returns the failed-attempt count)
FR-FAU-02	Detect a cabinet open too long, notify, close	PS-FAU-01	Cabinet Controller, Motor and Sensor Driver	cabinet_controller.py: Cabinet_Controller._monitor_cabinets, _check_open_cabinets (OPEN_WARNING_TIME, OPEN_FAULT_TIME, MONITOR_PERIOD); automatic close call in _check_open_cabinets [documented implemented; validation case VT-FAU-01] cabinet_motor_and_sensor_driver.py: CabinetMotorAndSensorDriver.close_cabinet
FR-FAU-03	Detect a failed open or close mechanism, notify	PS-FAU-02	Motor and Sensor Driver, Cabinet Controller	cabinet_motor_and_sensor_driver.py: CabinetMotorAndSensorDriver.get_cabinets_states, set_cabinets_states ("fault") cabinet_controller.py: Cabinet_Controller._check_medication_request (REJECTED for a cabinet in fault), dispense_medicine and add_medicine (undo when open fails), get_cabinet_states
FR-OPE-01	Standby, dispensing, locked and emergency shutdown modes	PS-OPE-01	Cabinet Controller	cabinet_controller.py: ALL_STATES, LOGIN_ALLOWED_IN, CAN_ENTER, NO_MEDICATION_ACCESS_STATES, Cabinet_Controller.change_state, _set_state, logout

Req ID	Requirement (short)	Precise spec	Module	Code (file: class.function)
FR-EXT-01	Provide dispensing, inventory and fault information to the Monitoring System	PS-EXT-01, PS-EXT-02	External Communication Driver, Serial Driver	external_communication_driver.py: ExternalCommunicationDriver.start, _run, _handle_message, _send_response, _get_auth_details, _set_auth_details; targets inventory, controller_state, events [planned] serial_driver.py: SerialDriver.open, send, read, read_all, add_rx_message, get_status
FR-INV-03 (new)	Max 100 units per medicine and 300 units per cabinet	PS-INV-02, PS-INV-03, PS-INV-05	Inventory Tracker	inventory_tracker.py: InventoryTracker.add_medicine (max_medicine_quantity, max_cabinet_quantity) config/config.yaml: inventory.max_medicine_quantity, inventory.max_cabinet_quantity
NFR-SEC-01	Prevent unauthenticated users from accessing medication	PS-AUT-01, PS-AUT-03	Authenticator, Cabinet Controller	cabinet_controller.py: Cabinet_Controller._check_medication_request (NOT_LOGGED_IN, CAN_ACCESS_MEDICATION, allowed medication), username_password_login authenticator.py: Authenticator.check_username_password
NFR-SEC-02	No access after 3 consecutive failed logins	PS-AUT-02	Authenticator	authenticator.py: Authenticator.check_username_password (LOCKED_OUT)
NFR-REL-01	Inventory levels monitored with 99% accuracy	PS-INV-04, PS-INV-06	Inventory Tracker	inventory_tracker.py: InventoryTracker._write_rows (temporary file, then os.replace), the lock in add_medicine, dispense_medicine and get_inventory; automated reference-ledger verification VT-INV-01 (1,000 trials; execution evidence pending)

Req ID	Requirement (short)	Precise spec	Module	Code (file: class.function)
NFR-REL-02	Fewer than 1 unplanned shutdown per 10,000 hours	PS-REL-01	Cabinet Controller, External Communication Driver	Design mapping (quantitative field verification pending): cabinet_controller.py: _locked (RLock), _monitor_cabinets (catches errors and keeps running) external_communication_driver.py: ExternalCommunicationDriver._run (a failed message does not stop the thread)
NFR-AVA-01	At least 99.9% availability in operating periods	PS-REL-01	Cabinet Controller, External Communication Driver	Design mapping (quantitative field verification pending): cabinet_controller.py: _locked, _monitor_cabinets external_communication_driver.py: ExternalCommunicationDriver._run (local operation does not depend on the link)
NFR-PER-01	Dispense within 5 s of selection and confirmation	PS-DIS-01	Cabinet Controller	cabinet_controller.py: Cabinet_Controller.dispense_medicine (no waits; inventory update then open_cabinet)
NFR-TRA-01	Dispensing traceable to user, medicine, quantity and time	PS-EVE-01	Event Logger, Cabinet Controller	cabinet_controller.py: Cabinet_Controller.dispense_medicine (log line with user, quantity, medicine, cabinet) event_logger.py: EventLogger (adds the time); files/events.log
NFR-USE-01	Routine operations with one hand	PS-USE-01	GUI	GUI/main_GUI.py [planned]
NFR-USE-02	Intuitive UI, minimal training	PS-USE-01	GUI	GUI/main_GUI.py [planned]

5.1 Requirement verification and implementation status

The file/class column above shows implementation location, not proof of requirement satisfaction. The table below separates observed/documented implementation from proposed verification. "Documented implemented" reflects the D2 update, while an unexecuted verification case is not reported as a passed test.

Requirement(s)	Status in this report	Verification reference / pass condition
FR-FAU-02 / PS-FAU-01	Automatic CLOSE on 180 s: documented implemented	VT-FAU-01: force sensor OPEN for 180 s, confirm one close command, Fault transition and fault-preserving logout; test execution not attached.
FR-DIS-01 / PS-DIS-01; FR-INV-01 / PS-INV-05	Transaction-level rollback specified	VT-INV-01: inject failed OPEN for dispense and restock; only the latest pending delta is undone.
NFR-REL-01 / PS-INV-06	Quantitative verification specified	VT-INV-01: >= 990/1,000 ledger comparisons match; zero failed-open rollback and stock invariant failures.
FR-EXT-01 / PS-EXT-01	Protocol partially implemented	VT-EXT-01: for supported requests on Ready link, capture response time <= 1 s; other targets remain planned.
NFR-REL-02; NFR-AVA-01	Reliability / availability design mechanisms present; targets not empirically demonstrated	VT-REL-01: gather operating-time and failure/recovery data; a 1,000-case inventory test does not establish 10,000 h reliability or 99.9% availability.
NFR-USE-01/02 / PS-USE-01	GUI planned	VT-USE-01: physical one-handed interaction and usability test after GUI implementation.

5.2 Automated 1,000-case inventory-accuracy test (VT-INV-01)

Objective and denominator: verify NFR-REL-01 as software-ledger accuracy, not physical medication-count accuracy, over exactly 1,000 independently checked transaction cases. Set a reproducible random seed and initialize a reference ledger with the same cabinet/medicine records as the ADC. In each iteration, generate one valid dispense, valid restock, invalid/over-capacity request, or simulated motor-open failure. Record the intended result in the reference ledger independently of the inventory implementation; apply the request to the system; then reopen the persisted inventory and compare all affected quantities, total-per-cabinet constraints and status. For motor-open failures, verify that the exact latest inventory delta was reversed and unrelated records remain unchanged. Include periodic restarts/readback to exercise persistence.

Measurement and acceptance: increment matches when the persisted state and returned status agree with the reference oracle. Accuracy = 100 x matches / 1,000 percent. At least 990 matching cases are required to meet the D1 threshold of 99%. Separately, all 1,000 trials must satisfy nonnegative quantities, 100-unit-per-medicine and 300-unit-per-cabinet limits, authorization guards, and correct rollback on simulated failed openings. Record mismatches, seed, software version and test log path. A threshold match must not conceal any safety-invariant failure.

Evidence status: this section specifies a reproducible automated test and its acceptance criteria; no executable project source or test-run output was provided with this report revision, so an observed pass count is not asserted. Attach the CI/unittest log and recorded match count after execution (for example: matches = [recorded result]/1,000).

5.3 Remaining quantitative targets

NFR-REL-02 (fewer than one unplanned shutdown per 10,000 operating hours) and NFR-AVA-01 (at least 99.9% availability) remain operational targets. Locking, persistence, and exception handling are supporting design controls but do not constitute measured proof. The implementation trace shows these mechanisms; verification requires a documented operating-time/failure log or defensible reliability analysis. The 1,000-case VT-INV-01 test must not be reused as evidence of availability.

In Section 5, [planned] denotes a designed interface or function not yet implemented in the report's source mapping. The automatic 180 s close path has been updated to documented implemented; this does not replace execution evidence. Where targets lack test results, the verification plan identifies the evidence still required.

6. Assumptions and Decisions

The following entries distinguish assumptions inherited from D1 from decisions refined or introduced for D2. Each identifies the design consequence, a rationale, and the affected requirements or modules. Hardware/clinical deployment assumptions must be validated outside this software prototype.

ID / change from D1	Assumption or decision	Rationale, privacy or accessibility impact	Affected requirement / module
AD-01 / D1 carried	The ADC contains motor-controlled medicine drawers.	Keep physical actuation behind a single swappable driver to avoid hardware dependencies throughout the controller.	FR-DIS-01; Motor/Sensor Driver
AD-02 / D1 carried	One-handed touchscreen input is the intended clinical interface.	Use single-point interaction and a short routine dispense flow; GUI remains planned in the prototype.	NFR-USE-01/02; PS-USE-01; GUI
AD-03 / D2 refined	Monitoring System initiates all exchanges; ADC answers but does not push data.	One polling authority makes inventory/lockout/fault synchronization predictable; when disconnected, ADC safety logic remains local.	FR-EXT-01, FR-INV-02; PS-EXT-01/02; External Communication

ID / change from D1	Assumption or decision	Rationale, privacy or accessibility impact	Affected requirement / module
AD-04 / D2 clarified	The physical card reader already decrypts cards and performs cryptographic/authenticity verification.	Only its trusted verified-identity output is accepted for username-only card login. This trust boundary is an external hardware assumption, not a GUI shortcut.	NAT-4/NAT-7; PS-AUT-01/02; Authenticator; card integration
AD-05 / D2 refined	Three staff roles and nurse-specific allowed-medication lists govern access.	Separate medicine access from administration and maintenance; apply role checks in Cabinet Controller.	FR-AUT-01, NFR-SEC-01; PS-AUT-03; Controller
AD-06 / D2 refined	Nurses cannot view full event logs.	Reduce exposure of sensitive event history while retaining audit access for authorized monitoring. Avoid passwords/patient identifiers in log text.	FR-EVE-01, NFR-TRA-01; Event Logger
AD-07 / D2 new	100 units per medicine and 300 units total per cabinet.	Enforce bounded simulated capacity with all-or-nothing restocking. Limits are software configuration assumptions pending hardware validation.	FR-INV-03; PS-INV-02/03/05; Inventory Tracker
AD-08 / D2 new	A failed drawer OPEN reverses the most recent pending transaction delta only.	Avoid counting a dispense/restock that did not gain access, while preserving unrelated previously committed records; log reversal failures as faults.	FR-DIS-01, NFR-REL-01; PS-DIS-01, PS-INV-05; Controller/Inventory

ID / change from D1	Assumption or decision	Rationale, privacy or accessibility impact	Affected requirement / module
AD-09 / D2 refined	180 s open detection triggers automatic CLOSE plus Fault.	Physical closure is attempted within one 1 s monitor cycle; if motor/sensor cannot confirm closure, the system remains Fault. The close call is documented implemented.	FR-FAU-02; PS-FAU-01; Controller/Motor Driver
AD-10 / D2 new	Logout preserves Locked, Fault and EmergencyShutdown states.	Ending a user session must not silently dismiss an unsafe physical condition; only authorized fault-clearing recovery may restore normal operation.	FR-OPE-01; PS-OPE-01; Controller
AD-11 / D2 refined	Serial responses must meet a uniform 1 s bound on a Ready link.	Align the software requirement, Monitoring System timeout, interface contract and test case; no bound is claimed while the link is disconnected.	FR-EXT-01; PS-EXT-01; External Communication/ Serial
AD-12 / D2 new	The prototype does not sense physical pill quantities.	Measure NFR-REL-01 as persisted ledger accuracy; independent physical reconciliation is a separate deployment requirement.	NAT-8; PS-INV-04/06; Inventory Tracker
AD-13 / D2 new	Use 1,000 automated reference-ledger checks for 99% accuracy.	A reproducible >=990/1,000 test gives a quantitative pass rule and separate zero-tolerance safety-invariant checks. Results require attached execution evidence.	NFR-REL-01; PS-INV-06; VT-INV-01
AD-14 / D2 refined	Store inventory in CSV with atomic replacement and serialization.	Keep persistence replaceable while preventing torn writes and simultaneous lost updates in the prototype.	PS-INV-04, PS-REL-01; Inventory Tracker

ID / change from D1	Assumption or decision	Rationale, privacy or accessibility impact	Affected requirement / module
AD-15 / D2 carried	A shared persistent event log records significant operations.	Centralize audit format and keep logging API stable if rotation, remote logging or storage changes.	FR-EVE-01; PS-EVE-01; Event Logger
AD-16 / D1 carried	The system should support future hardware and protocol replacements.	Separate protocol from link, inventory storage from controller policy, and drawer simulation from commands.	Section 3; Serial, External Communication, Motor, Inventory
AD-17 / D2 clarified	Plaintext password storage and a simulated/unfinished UI are prototype limitations.	Before clinical use, add salted password hashing, reader integration, authenticated management access, and verify usability/accessibility on real hardware.	NFR-SEC-01/02, NFR-USE-01/02; Authenticator, GUI

Citations

Anthropic. (2026). Claude (Opus 5.5 version) [Large language model]. https://claude.ai

OpenAI. (2026). *ChatGPT* (5.6-sol) [Large language model]. https://chatgpt.com